

Script-Controlled ACL: Record Access Security

Restricting Record Access Based on Dynamic Field Values in Enterprise Platforms

1. Overview & Objective

This technical specification defines the architecture and implementation guidelines for a script-controlled Access Control List (ACL) rule. The primary objective is to restrict record and field-level read/write permissions dynamically based on field values (e.g., `state`, `u_confidentiality`, or `u_assigned_department`).

By enforcing security at the database tier via server-side evaluation, this mechanism prevents unauthorized access across user interfaces, REST/SOAP API endpoints, and reporting engine queries.

2. Scope & Target Configuration

- **Target Table:** `[table_name]` (e.g., `sn_hr_core_case`, `incident`)
- **Operation:** `read` (or `write` / `create` / `delete`)
- **Type:** `record`
- **Target Level:**
 - **Table-Level (Row):** `[table_name]` — Controls access to the record itself.
 - **Field-Level (Column):** `[table_name].[field_name]` — Controls access to specific attributes.
- **Active:** `true` | **Advanced:** `true`

3. Evaluation Order & Performance Pipeline

To ensure optimal system performance and prevent unnecessary execution of JavaScript engine threads, ACL rules are evaluated using a strict three-tier execution hierarchy:



4. Implementation Script Structure

The script uses the global `answer` variable to grant (`true`) or deny (`false`) record access contextually.

```

/**
 * ACL Script: Restrict access based on field value logic.
 * Available Context: current (GlideRecord), gs (GlideSystem), answer (boolean)
 */
(function executeRule(current, previous) {

    answer = false; // Default deny

    if (!current || current.isNewRecord()) {
        answer = true; // Allow creation context
        return;
    }

    var currentUser = gs.getUserID();
    var fieldValue = current.getValue('u_confidentiality_level');

    switch (fieldValue) {
        case 'public':
            answer = true;
            break;

        case 'restricted':
            if (current.assigned_to == currentUser ||
                current.opened_by == currentUser ||
                gs.getUser().isMemberOf(current.getValue('assignment_group'))) {
                answer = true;
            }
            break;

        case 'highly_confidential':
            if (gs.hasRole('x_security_admin')) {
                answer = true;
            }
            break;

        default:
            answer = false;
            break;
    }

})(current, previous);

```

5. Verification & Test Matrix

Test ID	User Role	Field Value	User Context	Expected Result
TC-01	itil	public	Unrelated User	Access Granted
TC-02	itil	restricted	Assigned User	Access Granted
TC-03	itil	restricted	Unrelated User	Access Denied
TC-04	itil	highly_confidential	Group Member	Access Denied
TC-05	x_security_admin	highly_confidential	Unrelated User	Access Granted

6. Implementation & Operational Notes

- **List View Security Warnings:** Script-controlled ACLs evaluated on list views may generate "Security restricts record count" messages. Combine with a synchronized *Before Query Business Rule* to filter records at the database query level for optimized UX.
- **Impersonation Testing:** All security rules must be validated under non-admin impersonation across Form Views, List Views, and REST API calls.